Grammar

Grammar A grammar G is said to be a qudruple / 4-tuple

$G = (V, T, S, P)$

where V : a finite nonempty set of elements (these elements are called "Variables")

T : a finite nonempty set whose elements are called "terminals"

$S \in V$ is a special symbol called the start ~~symbol~~ variable / "Start symbol"

P : is a finite set of "productions" / "production rules"

$V \cap T = \Phi$

Ex: ① Consider the grammar $G = (\{S, B\}, \{a, b\}, S, P)$, with P given by

$$P = \begin{cases} S \to aSB & \text{("read as S derives aSB")} \\ S \to aB & \\ B \to b & \text{(read as B derives b)} \end{cases}$$

here, $V = \{S, B\}$, $T = \{a, b\}$

□ Getting a string $w \in L$ from this grammar is called derivation,

□ One such derivation is shown below.

You have to start with start symbol S, Now, the Language generated by G is $L = \{a^n b^n \mid n \geq 1\}$. To generate at

$S \Rightarrow aSB$ At every step, only one ~~of the~~ variables (here S and B) will be replaced by its right hand side, i.e, you can either replace S with aSB / aB or replace B with "b". If you start replacing leftmost variables first, then it is called left most derivation. If you " " " rightmost " " " " " " right most derivation, Here we follow leftmost derivation.

$S \Rightarrow aSB \ldots$
$\Rightarrow aaBB \ldots$
$\Rightarrow aabB \ldots$
$\Rightarrow aabb$

This entire procedure is called derivation. and whatever intermediate string we got in the way are called Sentential forms/sequential form

The strings which contain variables as well as terminals, are called "sentential forms" of the derivation

Another representation of getting derivation is called "Derivation tree")

Now, if we traverse this derivation tree, so, starting from "S", I got "a" first, and then another "a" and then "b" and followed by another "b"

$\boxed{aabb}$ → This is called yield of the parse tree or yield of " derivation tree.

(*)

Ex: (2) For a Language of set of all strings of length 2, Construct a grammar

$$L = \{aa, ab, ba, bb\}$$

The grammar G is as follows.

$$S \to aa \mid ab \mid ba \mid bb$$

~~or~~ If the Language is finite, then it is very easy to construct a grammar.

Alternate way The RE for $L = \{aa, ab, ba, bb\}$ is $(a+b)(a+b)$

Hence, we can write production rules as follows

$$S \to AA$$
$$A \to a \mid b$$

('+' means 'union' or 'or' So, A derives a or A derives b)

this production generates a+b

(*)

Ex: (3) Find a grammar that generates $L = \{a^n \mid n \geq 0\}$

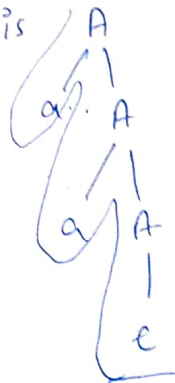
$L = \{\epsilon, a, aa, aaa, \ldots\ldots\}$

$\boxed{A \to aA \mid \epsilon}$ or $\boxed{A \to Aa \mid \epsilon}$

It is like, RE = a^* (i.e, ~~positive~~ Kleene closure of A)

Now, the parse tree for a is

the parse tree for aa is

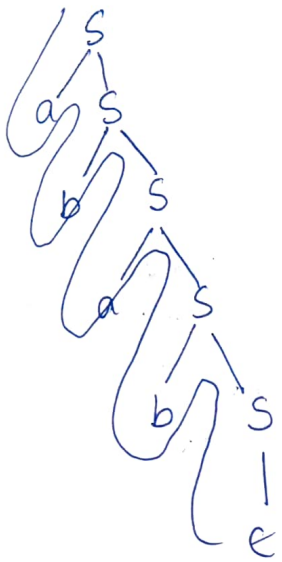


$aa\epsilon = \boxed{aa}$

Ex: (4) Find a grammar that generates RE $(a+b)^*$.

Ans $S \to aS \mid bS \mid \epsilon$ (go to either generating "a" or "b" or "ϵ").

Now, to generate the string $\omega = abab$, the parse tree is as follows



$\boxed{abab}$ yield of the derivation tree

(*) Ex: (5) Find a grammar G, that gerates language whose string length is at least 2.

Ans $L = \{aa, ab, ba, bb, aaa, \ldots bbb, aaaa \ldots\}$

Now, RE for $L = (a+b)(a+b)(a+b)^*$

$S \to AAB$
$A \to a/b$
$B \to aB \mid bB \mid \epsilon$ this production is for generation $(a+b)^*$ as shown in Ex (4)

Q How to produce the ~~building block~~ grammar for this language?

Ans See the Building block, here $(a+b)(a+b)$ represents 2 length strings,
So, one A can generate $(a+b)$
another A " " another $(a+b)$
and we already know how to generate $(a+b)^*$ by using $B \to aB \mid bB \mid \epsilon$

∴ final production is
$S \to AAB$
$A \to a \mid b$
$B \to aB \mid bB \mid \epsilon$

Ex: (6) Find a grammar G that generates Language over $\{a, b\}$ where $|w|$ is at most 2.

Ans R.E. $(a+b+\epsilon)(a+b+\epsilon)$

$S \to AA$

$A \to a|b|\epsilon$ This production generates $(a+b+\epsilon)$

Ex: (7) Find a grammar G that generates the Language over $\{a, b\}$ where each string starts with a and ends with b

Ans $RE = a(a+b)^* b$

$S \to aAb$

$A \to aA|bA|\epsilon$

Ex: (7) Find a grammar G for set of all strings starting and ending with different symbols.

Ans $RE = a(a+b)^* b + b(a+b)^* a$

$S \to aAb \mid bAa$

$A \to aA \mid bA \mid \epsilon$

Ex: (8) Find a grammar G for set of all strings starting and ending with same symbol

Ans $RE = a(a+b)^* a + b(a+b)^* b + a + b + \epsilon$

$S \to aAa \mid bAb \mid a \mid b \mid \epsilon$

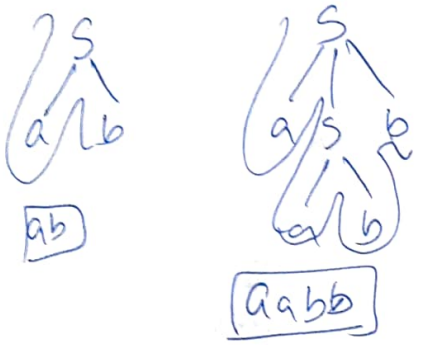
$A \to aA \mid bA \mid \epsilon$

Ex: (9) Find a grammar G that generates $L = \{a^n b^n \mid n \geq 1\}$

Ans Here L is not regular, So, we can't analyze RE to generate grammar. Here, we follow different approach.

$S \to aSb \mid ab$

This production is used to stop the derivation



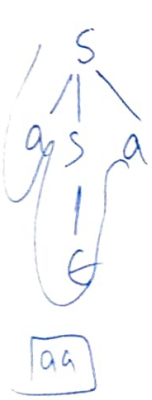
Ex: (10) Find a grammar of all palindromes =

Find a grammar G for set of even length palindromes $L_1 = \{ww^R \mid w \in \{a,b\}^*\}$ or set of odd ~~length~~ Length palindromes $L_2 = \{waw^R \mid w \in \{a,b\}^*\}$ or $L_3 = \{wbw^R \mid w \in \{a,b\}^*\}$

So, basically we need a Language of all the palindromes,

$L = L_1 \cup L_2 \cup L_3 = ww^R \cup waw^R \cup wbw^R$

Ans $S \to aSa \mid bSb \mid a \mid b \mid \epsilon$



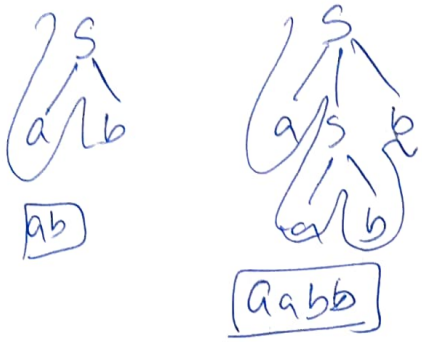
(*)

Ex: (9) Find a grammar G that generates $L = \{a^n b^n / n \geq 1\}$

Ans Here L is not regular, So, we can't analyze RE to generate grammar. Here, we follow different approach.

$S \to aSb \mid ab$

This production is used to stop the derivation



(*) Find a grammar of all palindromes =

Ex. (10) Find a grammar G for set of even length palindromes

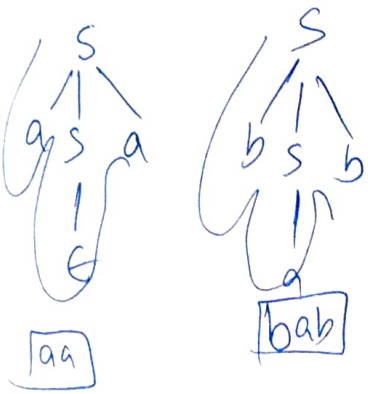
$L_1 = \{ww^R / w \in \{a,b\}^*\}$ or set of odd ~~length~~ Length palindromes

$L_2 = \{waw^R / w \in \{a,b\}^*\}$ or $L_3 = \{wbw^R / w \in \{a,b\}^*\}$

So, basically we need a Language of all the palindromes,

$L = L_1 \cup L_2 \cup L_3 = ww^R \cup waw^R \cup wbw^R$

Ans $S \to aSa \mid bSb \mid a \mid b \mid \epsilon$



Ex: (11) Find a grammar G that generates set of all even length strings

Ans R.E = $((a+b)(a+b))^*$

$S \to BS \mid \epsilon$
$B \to AA$
$A \to a \mid b$

OR

$S \to AAS \mid \epsilon$
$A \to a \mid b$

Ex: (12) G for $L = \{a^n b^m \mid n, m \geq 1\}$ Here, first we generate a^n, then we generate b^m independently, this is different from $\{a^n b^n \mid n \geq 1\}$,

Ans

$S \to AB$
$A \to aA \mid a$ → This one generates a^n
$B \to bB \mid b$ → This " " b^m

Then we club this two by $S \to AB$

Ex: (13) Grammar G for $L = \{a^n b^n c^m \mid n, m \geq 1\}$

Unlike Ex: (12) Here, you cannot generate a^n, b^n and c^m individually, because there are dependency between $n_a(w)$ and $n_b(w)$

generate first a's and b's together using production,

$A \to aAb \mid ab$

then use a production $B \to cB \mid c$, to generate c^m

then use $S \to AB$

∴ final production rules are

$S \to AB$
$A \to aAb \mid ab$
$B \to cB \mid c$

Ex: (14) G for $L = \{a^n c^m b^n \mid n, m \geq 1\}$

$S \to aSb \mid aAb$

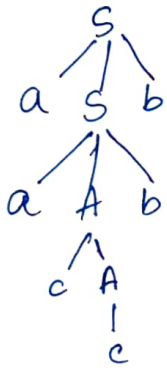
$A \to cA \mid c$

This will any no of c's like c^m

Here, a's and b's have to be generate together because they have to be matched and c's has to be generated separately and than we have to put c's in the middle.

Note: Here, we can use $S \to aSb$ as many times as we want to generate $a^n c^m b^n$

If we want a string, $w = aacbb$, then



and whenever you want to stop it, stop it with a single "c" or any no c's using $A \to cA \mid c$

Note: Here $S \to aSb \mid A$, $A \to cA \mid c$ X is not a valid grammar, because If we choose $S \Rightarrow A \Rightarrow cA \Rightarrow cc$ then we get only c's without having any a's or b's.

Q Ex: Grammar for odd length palindromes i.e, $waw^R \cup wbw^R$ where $w \in \{a,b\}^*$

Ans $S \to aSa \mid bSb \mid a \mid b$

Here aSa generates all strings which are same from forward and backward direction, Similarly bSb does the same thing. If we replace S by "a" or "b", we get odd length palindromes like waw^R or wbw^R.

Q Ex: Grammar for even length palindromes i.e, ww^R

Ans $S \to aSa \mid bSb \mid \epsilon$

Ex: (15) ⊛

Grammar G for $L = \{a^n b^n c^m d^m \mid n, m \geq 1\}$

Ans

$S \to AB$

$A \to aAb \mid ab$ This will generate $a^n b^n \mid n \geq 1$

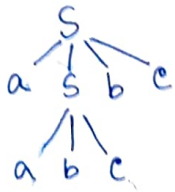
$B \to cBd \mid cd$ " " " $c^n d^n \mid n \geq 1$

Then we club or merge those two

Ex: ⊛ (16) Construct a grammar G for $L = \{a^n b^n c^n \mid n \geq 1\}$

Ans How to generate such grammar ?

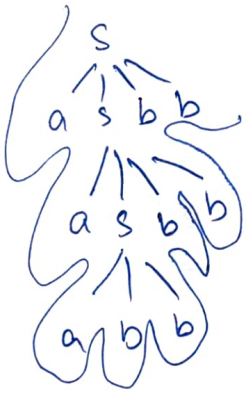
Let us take, $S \to aSbc \mid abc$, then



$\therefore aabcbc \cong a^n(bc)^n$, So, it is difficult to construct a such a context sensitive grammar. (See Push down Automata)

Ex: (17) $L = \{a^n b^{2n} \mid n \geq 1\}$

$S \rightarrow aSbb \mid abb$



$\boxed{a^3 b^6 = aaabbbbbb}$

(*)

Ex: (18) $L = \{a^n b^m c^m d^n \mid n, m \geq 1\}$

$S \rightarrow aSd \mid aAd$

$A \rightarrow bAc \mid bc$ This production generates $b^m c^m \mid m \geq 1$

Ex: 19 $L = \{a^n b^m c^n d^m \mid n, m \geq 1\}$

Ans Here a's and c's has to be matched and b's and d's has to be matched. Any content free grammer can't be constructed for this Language (See Push down Automata)

Ex: 20 * $L = \{a^{m+n} b^m c^n \mid m, n \geq 1\} = \{a^n a^m b^m c^n \mid n, m \geq 1\}$

$S \to aSc \mid aAc$

$A \to aAb \mid ab$

Ex. (*) (21) $L = \{a^n b^{n+m} c^m \mid n, m \geq 1\}$

$= \{a^n b^n b^m c^m \mid n, m \geq 1\}$

$S \to AB$

$A \to aAb \mid ab$

$B \to bBc \mid bc$

Ex: (22) $L = \{a^n b^m c^{n+m} \mid n, m \geq 1\}$

i.e, $\{a^n b^m c^m c^n \mid n, m \geq 1\}$

Ans $S \to aSc \mid aAc$

$A \to bAc \mid bc$

Note: A RL (Regular Language) can be described by three ways

(i) FA

(ii) RE

(iii) RG

Classification of Grammars

According to Chomsky there are 4 types of grammars →

↳ Type 3 / Regular Grammar / Most Restrictive Grammar

~~If the grammar has all the productions of the form~~

* A "regular grammar" is one that is either left-linear or right linear.

* A grammar $G = (V, T, S, P)$ is said to be "right-linear" if all productions are of the form

$$\boxed{A \to \alpha B \mid \beta}$$

where $A, B \in V$, and $\alpha, \beta \in T^*$. Here, α and β are purely terminals and they can even be ϵ. Here, A and B are purely variable. Here the variable is present at the rightmost side.

* A grammar $G = (V, T, S, P)$ is said to be "left-linear" if all the productions are of the form

$$\boxed{A \to B\alpha \mid \beta}$$

where $A, B \in V$ and $\alpha, \beta \in T^*$. Here the variable is present at the leftmost side

Ex: ① The grammar $G_1 = (\{S\}, \{a, b\}, S, P_1)$ with P_1 given as

$$S \to abS \mid a$$

is right linear.

The grammar $G_2 = (\{S, S_1, S_2\}, \{a, b\}, S, P_2)$ with P_2 given as

$$S \to S_1 ab$$
$$S_1 \to S_1 ab \mid S_2$$
$$S_2 \to a$$

is left linear.

□ Both G_1 and G_2 are regular grammar.

Ex: ② The grammar $G = (\{S, A, B\}, \{a, b\}, S, P)$ with productions $S \to A$, $A \to aB \mid \lambda$, $B \to Ab$ is not regular. Although every production is either left-linear or right-linear form, the grammar itself is ~~not~~ neither right-linear or left-linear, and therefore not regular. The grammar is an example of linear grammar.

□ "linear grammar": is a grammar in which at most one variable can occur on the right side of any production, without restriction on the position of this variable.

Note: A RG is always linear, but not all linear grammars are regular.

□ ↳ Type 2 grammar / Context Free grammar (CFG)

A grammar $G = (V, T, S, P)$ is said to be "CFG" if all productions are of the form

$$\boxed{A \to \alpha}$$

where $A \in V$, $\alpha \in (V \cup T)^*$. Here α is a string of terminals and variables.

Ex: ① The grammar $G = (\{S\}, \{a, b\}, S, P)$ with productions

$S \to aSa$

$S \to bSb$

$S \to \epsilon$

is Context free. A typical derivation in this grammar is

$$S \Rightarrow aSa \Rightarrow aaSaa \Rightarrow aabSbaa \Rightarrow aab\epsilon baa$$
$$\Rightarrow \underbrace{aab}_{w}\underbrace{baa}_{w^R}$$

This and similar derivations, show that the Language L generated by this grammar G is as follow

$$L(G) = \{ ww^R : w \in \{a, b\}^* \}$$

Q Why Context Free? Ans No dependence on Surrounding symbol: The replacement of a non-terminal A depends solely on A itself and not on any symbols (context) around it. Hence it is "free" from the context of the surrounding symbols.

Type-1 grammar or CSG (Context Sensitive Grammar)

A grammar $G = (V, T, S, P)$ is said to be "Context sensitive" grammar if all the productions are of the form

$$\boxed{\alpha A \beta \rightarrow \alpha \gamma \beta}$$

where $A \in V$, $\gamma \in (V \cup T)^+$ and $\alpha, \beta \in (V \cup T)^*$ and $\boxed{|\alpha A \beta| \leq |\alpha \gamma \beta|}$

Alternative definition

A grammar G is said to be CSG if all the productions are of the form

$$\boxed{x \rightarrow y}$$

where, $x, y \in (V \cup T)^+$ and $|x| \leq |y|$

(*) This grammar is "non contracting" in the sense that the length of successive sentential forms can never ~~be~~ decrease.

Q Why context sensitive ?

Ans Dependence on Surrounding symbols : Unlike CFG, where a non-terminal can be replaced regardless of it surrounding, CSG require the presence of the context (specific symbols, α and β) around the non-terminal for a production rule to be applied.

Q Does it have length preserving property ?

Ans Yes. The length of string on L.H.S. $(\alpha A \beta)$ is $\leq$ the length of string on R.H.S $(\alpha \gamma \beta)$

Type-0 / Unrestricted Grammar (This grammar generates Recursively enumerable Languages (the Languages accepted by TM))

A grammar $G = (V, T, S, P)$ is said to be type-0 grammar if all the productions are of the form

$$\boxed{x \to y}$$

where $x \in (V \cup T)^+$ and $y \in (V \cup T)^*$

The L.H.S of the production can contain any no of variables and terminals, but only restriction is that it can not be empty, i.e, it can't contain ϵ.

The R.H.S of the production is completely unrestricted.

Q Why Unrestricted?

Ans Because they have no restriction on the form of their production rules. Unlike other grammars (CSG, CFG or RG), unrestricted grammars can have any kind of production rule, allowing them to become most powerful type of grammar in the Chomsky hierarchy.